Vector Processors

Huiyang Zhou

Top-down approach

- Vector programs
- Vector ISA
- Vector Architecture
  - Vector instruction execution
  - Single lane vs. parallel lanes
  - Pipeline hazards
- Performance Evaluation/Estimation

- Difference from SIMT/GPU architecture

Vector program

- Y = a * X + Y, where X, Y are vectors and a is a scalar number

- Scalar program

```
for (int i = 0; i < 64; i++)
    Y[i] += a*X[i];
```

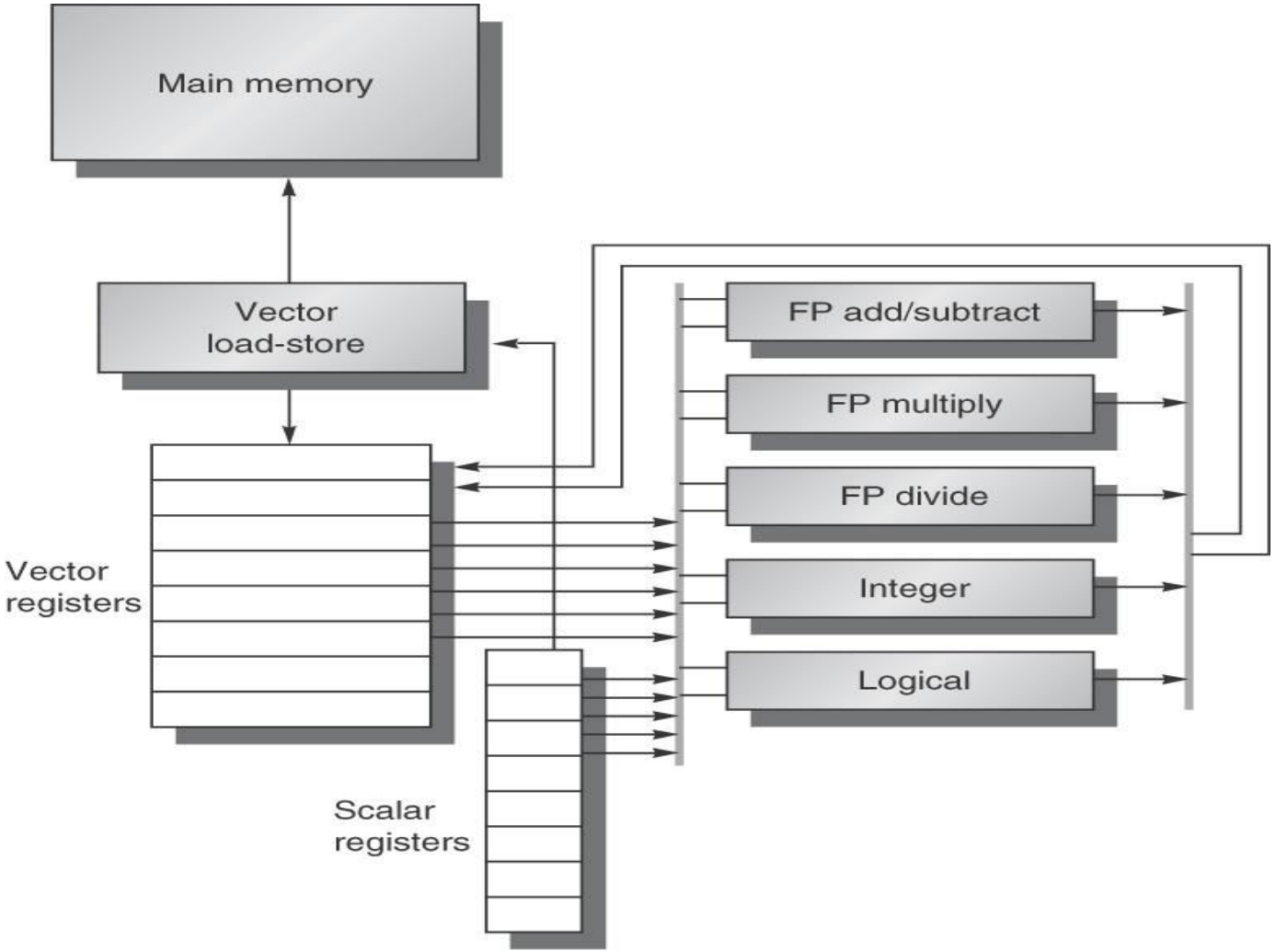
- Vector program

```
Y[0:63] = a*X[0:63] + Y[0:63]
```

Scalar (MIPS) code

```
L.D F0,a                ;load scalar a
DADDIU R4,Rx,#512       ;last address to load
Loop: L.D F2,0(Rx)      ;load X(i)
MUL.D F2,F2,F0          ;a × X(i)
L.D F4,0(Ry)            ;load Y(i)
ADD.D F4,F4,F2          ;a × X(i) + Y(i)
S.D 0(Ry),F4            ;store into Y(i)
DADDIU Rx,Rx,#8         ;increment index to X
DADDIU Ry,Ry,#8         ;increment index to Y
DSUBU R20,R4,Rx         ;compute bound
BNEZ R20,Loop           ;check if done
```

Vector Processor Architecture (VMIPS)



© 2007 Elsevier, Inc. All rights reserved.

Vector ISA: VMIPS

- Example architecture: VMIPS
  - Loosely based on Cray-1
  - Vector registers
    - Each register holds a 64-element, 64 bits/element vector
    - Register file has 16 read ports and 8 write ports
  - Vector functional units
    - Fully pipelined
    - Data and control hazards are detected
  - Vector load-store unit
    - Fully pipelined
    - One word per clock cycle after initial latency
  - Scalar registers
    - 32 general-purpose registers
    - 32 floating-point registers

Vector ISA: VMIPS

- Instructions
  - ADDVV.D, SUBVV.D
  - ADDVS.D, SUBVS.D, SUBSV.D
  - MULVV.D, DIVVV.D
  - MULVS.D, DIVVS.D, DIVSV.D
  - LV, SV
  - LVWS, SVWS (with stride) for strided accesses
  - LVI, SVI (indirect) for gather and scatter

Vector Code

Here is the VMIPS code for DAXPY.

```
L.D F0,a              ;load scalar a
LV V1,Rx              ;load vector X
MULVS.D V2,V1,F0 ;vector-scalar multiply
LV V3,Ry               ;load vector Y
ADDVV.D V4,V2,V3 ;add
SV Ry,V4              ;store the result
```

- The differences?
  - Dynamic instruction count
  - Pipeline stalls due to data dependencies

RISC-V Vector Extension

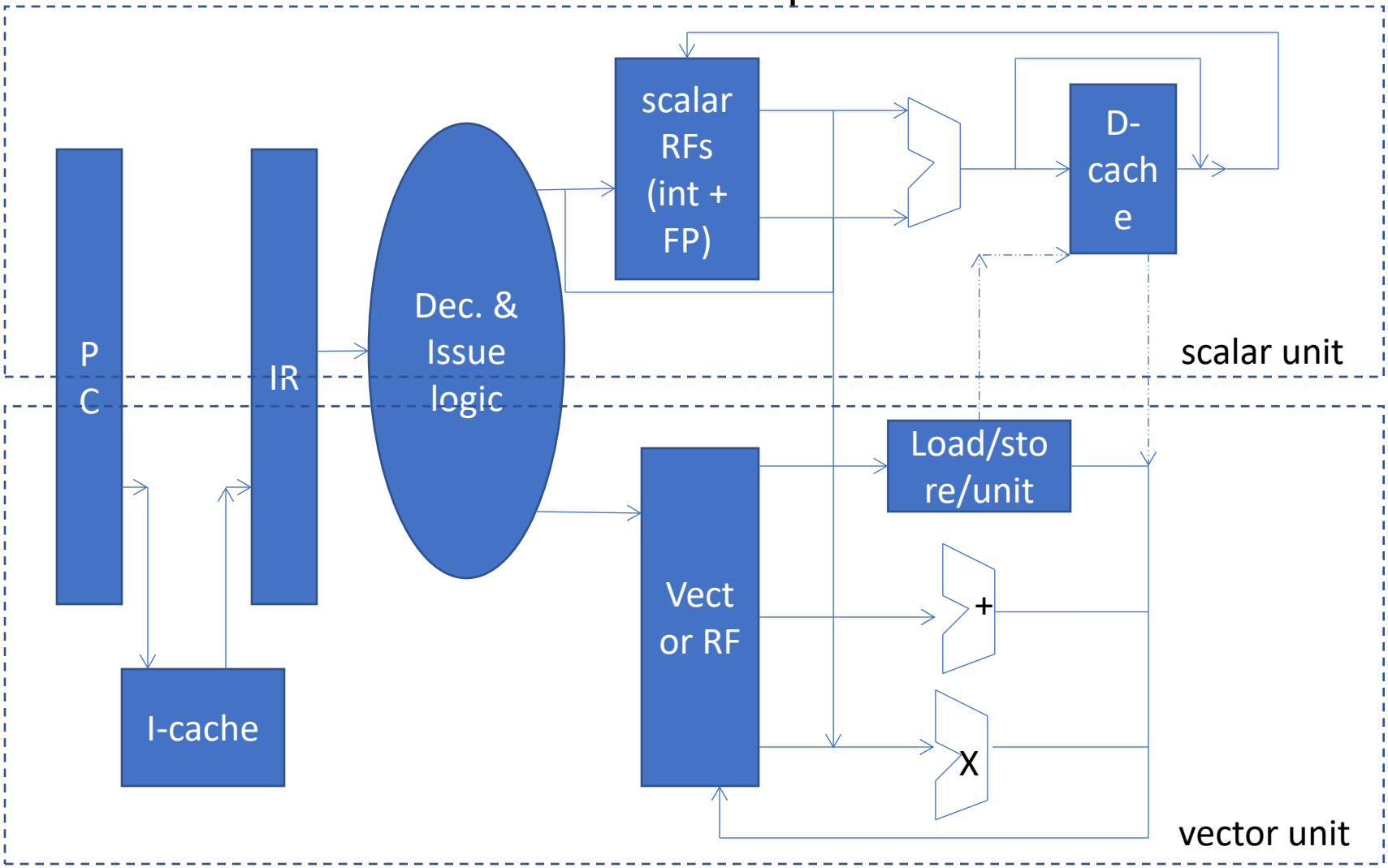
- https://github.com/riscv/riscv-v-spec/blob/master/v-spec.adoc

Vector register organization

- The vector extension adds 32 architectural vector registers, v0-v31 to the base scalar RISC-V ISA.
- Each vector register has a fixed VLEN bits of state. The base "V" vector extension requires that VLEN≥128.

- The vector length multiplier, *LMUL*, when greater than 1, represents the default number of vector registers that are combined to form a vector register group.
- The value in vsew sets the dynamic *selected element width* (SEW). By default, a vector register is viewed as being divided into VLEN/SEW elements.

Microarchitecture of vector processors



Pipeline execution

- Scalar vs. vector execution
- In-order vs. OOO
  - Diversified pipelines with data hazard checking logic
- Single lane vs. parallel lanes

- Detailed examples

- Notes
  - The scoreboard used in GPU architecture enables multiple instructions from a single warp to get in the execution pipelines
  - In scalar/vector pipelines, there are always multiple instructions from the same thread getting into the execution pipelines.

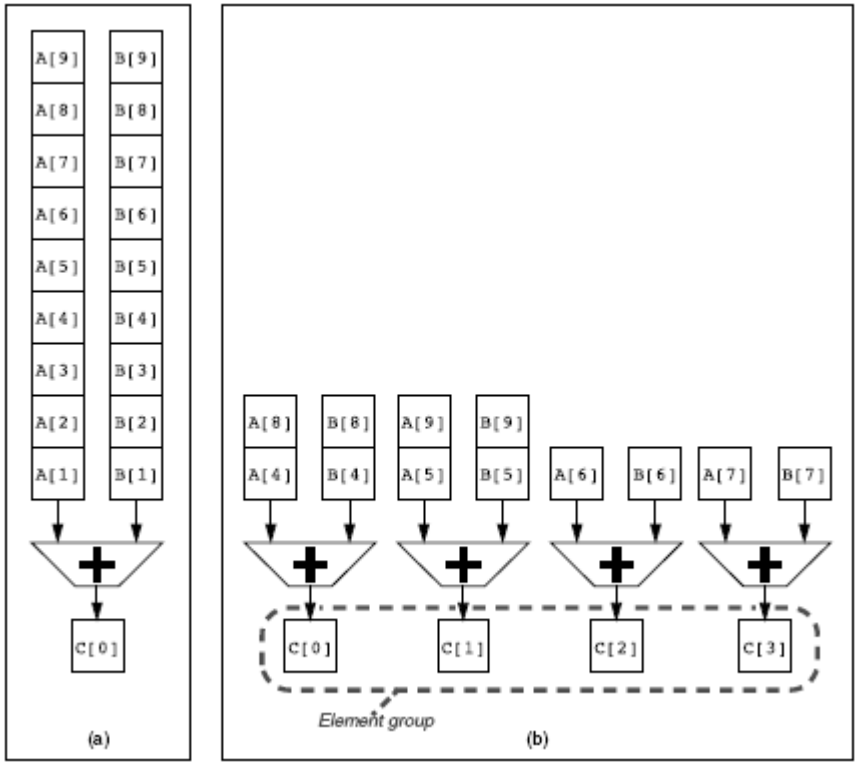
Challenges

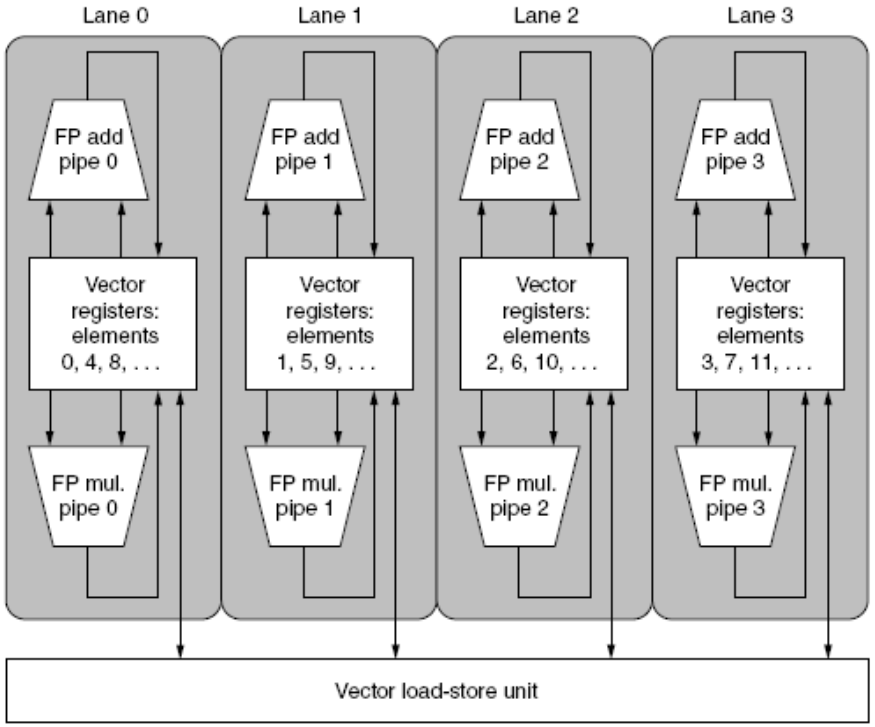
- Start up time
  - Latency of vector functional unit
  - Assume the same as Cray-1
    - Floating-point add => 6 clock cycles
    - Floating-point multiply => 7 clock cycles
    - Floating-point divide => 20 clock cycles
    - Vector load => 12 clock cycles
- Improvements:
  - > 1 element per clock cycle
  - Non-64 wide vectors
  - IF statements in vector code
  - Memory system optimizations to support vector processors
  - Multiple dimensional matrices
  - Sparse matrices
  - Programming a vector computer

Copyright © 2012, Elsevier Inc. All rights reserved.

Multiple Lanes

- Element *n* of vector register *A* is "hardwired" to element *n* of vector register *B*
  - Allows for multiple hardware lanes





Copyright © 2012, Elsevier Inc. All rights reserved.

Vector Length Control (1/2)

- Problem: In the following code, where n is a variable determined at the run-time.

```
for (i = 1, i < n; i++)
        Y(i) = a * X(i) + Y(i)
```

- Solution: A vector length control register (VLR)

- What if n is larger than the maximum vector length (MVL)?

Vector Length Control (2/2)

low = 1

VL = (n mod MVL) /*find the odd-size piece*/

for (j = 0, j < (n / MVL); j++) {/*outer loop*/

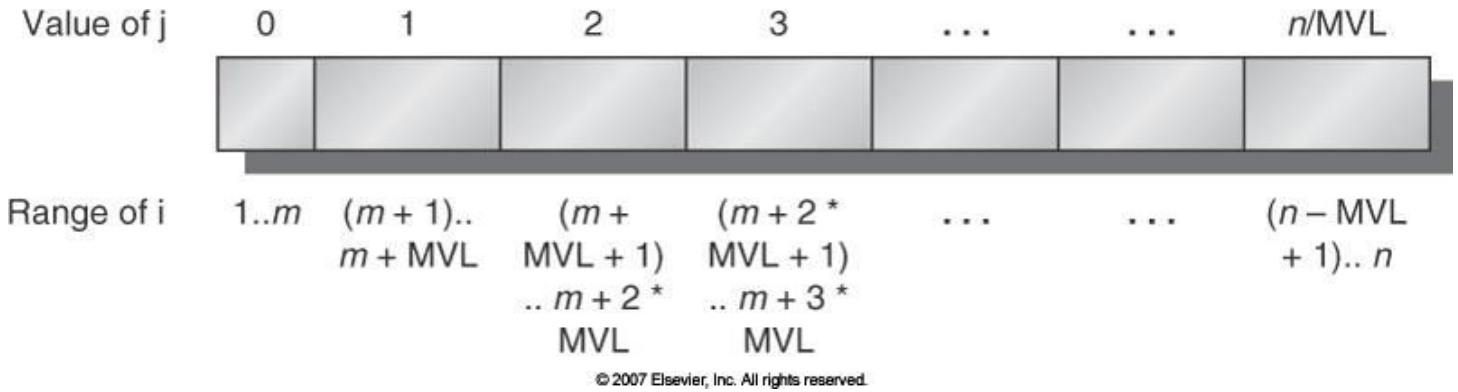
for(i = low, low + VL – 1; i++) /*runs for length VL*/

Y(i) = a * X(i) + Y(i) /*main operation*/

low = low + VL /*start of next vector*/

VL = MVL /*reset the length to max*/

}



© 2007 Elsevier, Inc. All rights reserved.

GPU solution?

Enhancing Vector Performance: Chaining

- Problem: flow dependency among vector instructions

  MULV.D V1,V2,V3

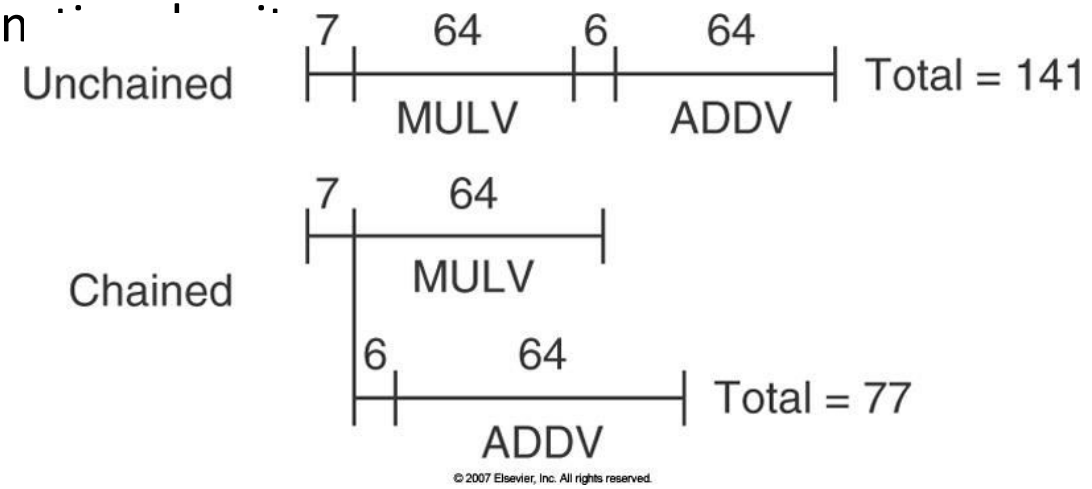
  ADDV.D V4,V1,V5

- Observation: Only dependency between the same elements in a vector register.

- Solution: Chaining

Chaining

- Chaining allows a vector operation to start as soon as the individual elements of its vector source operand become available: The results from the first functional unit in the chain are “forwarded” to the second fun



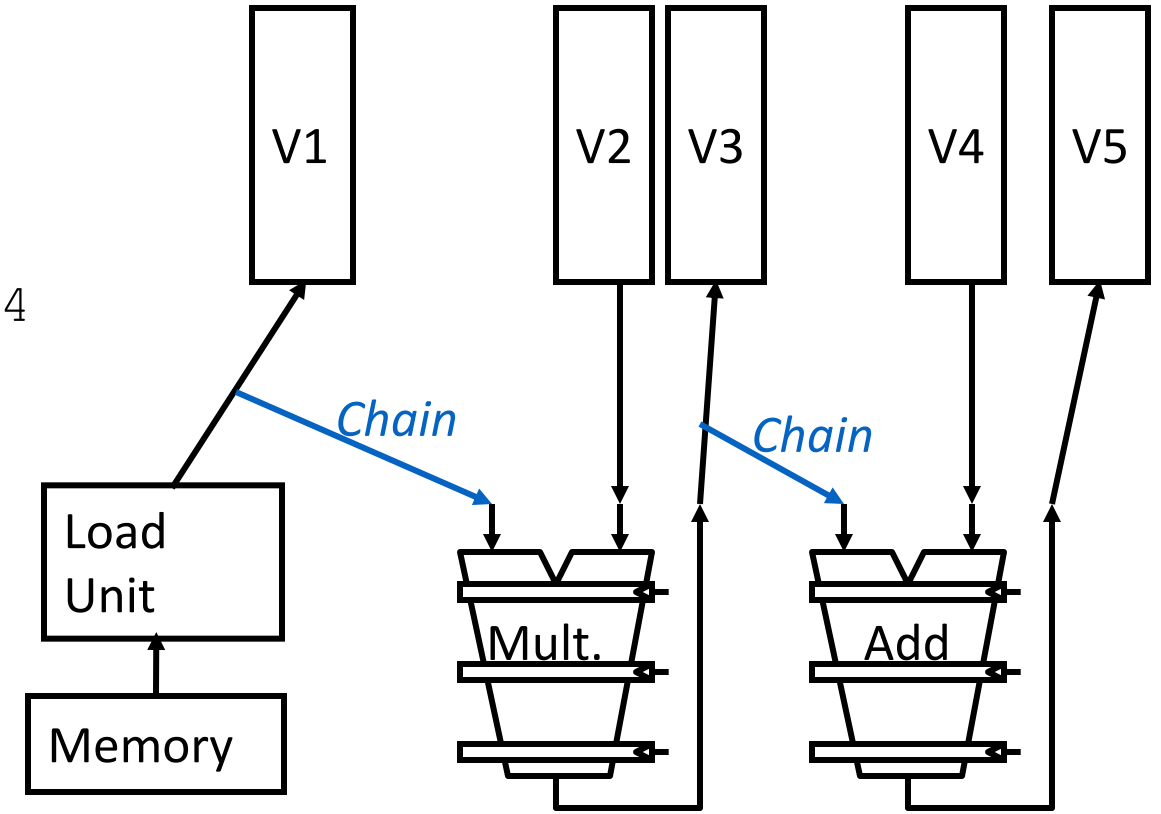
© 2007 Elsevier, Inc. All rights reserved.

GPU solution?

Vector Chaining

- Vector version of register bypassing
  - introduced with Cray-1

```
LV   v1
MULV v3,v1,v2
ADDV v5,v3, v4
```



Predication

- Problem:

```
for (i = 0; i < 64; i++)
    if (A(i)!= 0) then
        A(i) = A(i) – B(i)
```

- Solution: vector mask

```
LV V1,Ra          ;load vector A into V1
LV V2,Rb          ;load vector B
L.D F0,#0         ;load FP zero into F0
SNEVS.D V1,F0     ;sets VM(i) to 1 if V1(i)!=F0
SUBV.D V1,V1,V2   ;subtract under vector mask
CVM               ;set the vector mask to all 1s
SV Ra,V1          ;store the result in A
```

GPU solution?

Memory Banks

- Memory system must be designed to support high bandwidth for vector loads and stores
- Spread accesses across multiple banks
  - Control bank addresses independently
  - Load or store non sequential words
  - Support multiple vector processors sharing the same memory
- Example:
  - 32 processors, each generating 4 loads and 2 stores/cycle
  - Processor cycle time is 2.167 ns, SRAM cycle time is 15 ns
  - How many memory banks needed?

Copyright © 2012, Elsevier Inc. All rights reserved.

Stride

- Consider:

```
for (i = 0; i < 100; i=i+1)
        for (j = 0; j < 100; j=j+1) {
                A[i][j] = 0.0;
                for (k = 0; k < 100; k=k+1)
                A[i][j] = A[i][j] + B[i][k] * D[k][j];
        }
```

- Must vectorize multiplication of rows of B with columns of D
- Use *non-unit stride*
- Bank conflict (stall) occurs when the same bank is hit faster than bank busy time:
  - #banks / LCM(stride,#banks) < bank busy time

Copyright © 2012, Elsevier Inc. All rights reserved.

Scatter-Gather

- Consider:

```
for (i = 0; i < n; i=i+1)
        A[K[i]] = A[K[i]] + C[M[i]];
```

- Use index vector:

```
LV        Vk, Rk          ;load K
LVI       Va, (Ra+Vk)     ;load A[K[]]
LV        Vm, Rm          ;load M
LVI       Vc, (Rc+Vm)     ;load C[M[]]
ADDVV.D   Va, Va, Vc      ;add them
SVI       (Ra+Vk), Va     ;store A[K[]]
```

Copyright © 2012, Elsevier Inc. All rights reserved.

Vector Reductions

Problem: Loop-carried dependence on reduction variables

```
sum = 0;
for (i=0; i<N; i++)
    sum += A[i];  # Loop-carried dependence on sum
```

Solution: Re-associate operations if possible, use binary tree to perform reduction

```
# Rearrange as:
sum[0:VL-1] = 0                     # Vector of VL partial sums
for(i=0; i<N; i+=VL)                # Stripmine VL-sized chunks
    sum[0:VL-1] += A[i:i+VL-1]; # Vector sum
# Now have VL partial sums in one vector register
do {
    VL = VL/2;                        # Halve vector length
    sum[0:VL-1] += sum[VL:2*VL-1] # Halve no. of partials
} while (VL>1)
```

Pipeline Execution Example

```
Load      Rx, #0(0x400)     ;Read vector address
Load      Ry, #4(0x400)     ;Read vector address
LV        V1,Rx             ;load vector X
MULVS.D   V2,V1,F0    ;vector-scalar multiply
LV        V3,Ry             ;load vector Y
ADDVV.D   V4,V2,V3    ;add two vectors
SV        Ry,V4             ;store the sum
```

VMIPS processor pipeline:
8 ALU and MEM lanes, with chaining, all loads hit in L1 cache (1 cycle latency)
MUL 2 cycle latency

	Load	Rx, #0(0x400)	IF	ID	EX	MEM	WB														
	Load	Ry, #4(0x400)		IF	ID	EX	MEM	WB													
	LV	V1, Rx			IF	ID	s	EX	MEM	WB									0~7		
									EX	MEM	WB								8~15		
										EX	MEM	WB							16~23		
											EX	MEM	WB						24~31		
												EX	MEM	WB					32~39		
													EX	MEM	WB				40~47		
														EX	MEM	WB			48~55		
															EX	MEM	WB		56~63		
	MULVS.D	V2,V1,F0				IF		ID	s	E1	E2	WB									
											E1	E2	WB								
												E1	E2	WB							
													E1	E2	WB						
														E1	E2	WB					
															E1	E2	WB				
																E1	E2	WB			
																	E1	E2	WB		
																		E1	E2	WB	
	LV	V3, Ry						IF		ID	s	s	s	s	s	EX	MEM	WB			
											structural hazard						EX	MEM	WB		
	ADDVV.D	V4,V2,V3																			
	SV	Ry, V4																			

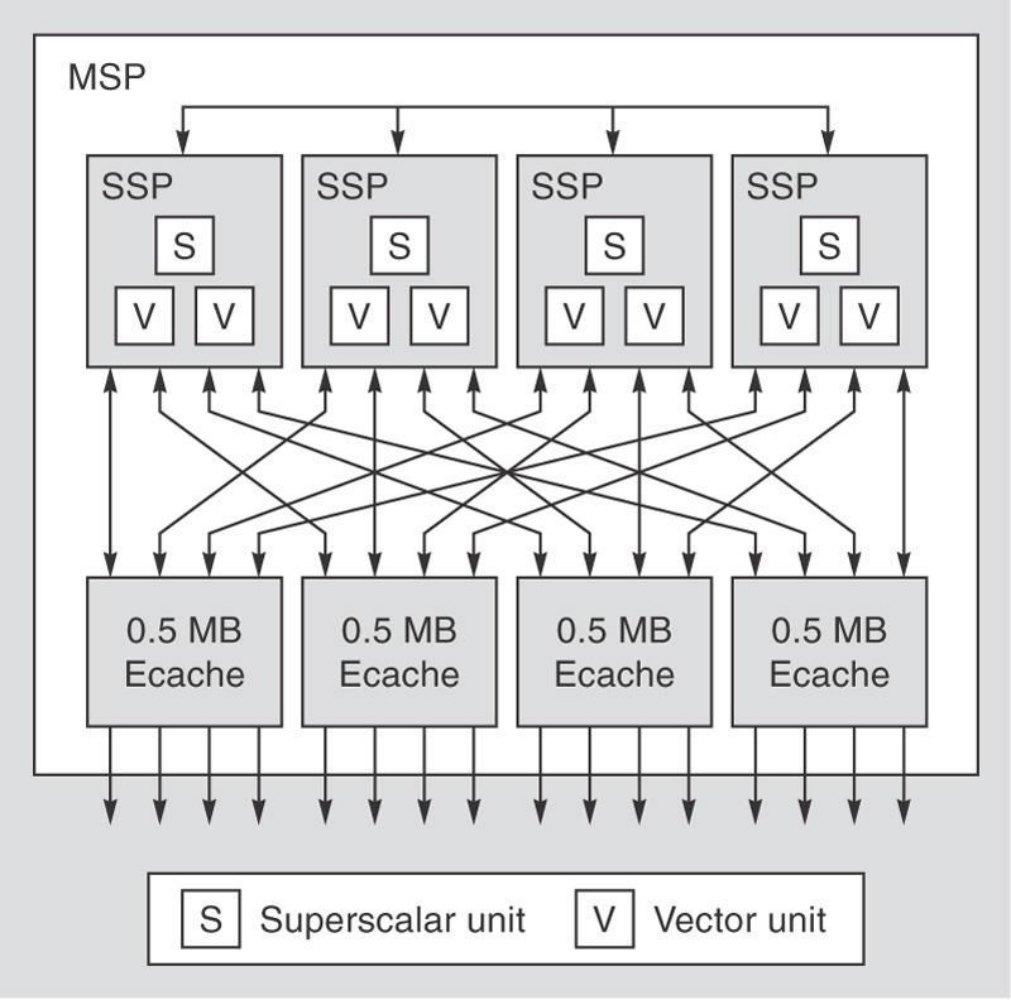
Programming Vec. Architectures

- Compilers can provide feedback to programmers
- Prog

Benchmark name	Operations executed in vector mode, compiler-optimized	Operations executed in vector mode, with programmer aid	Speedup from hint optimization
BDNA	96.1%	97.2%	1.52
MG3D	95.1%	94.5%	1.00
FLO52	91.5%	88.7%	N/A
ARC3D	91.1%	92.0%	1.01
SPEC77	90.3%	90.4%	1.07
MDG	87.7%	94.2%	1.49
TRFD	69.8%	73.7%	1.67
DYFESM	68.8%	65.6%	N/A
ADM	42.9%	59.6%	3.60
OCEAN	42.8%	91.2%	3.92
TRACK	14.4%	54.6%	2.52
SPICE	11.5%	79.9%	4.06
QCD	4.2%	75.1%	2.15

Copyright © 2012, Elsevier Inc. All rights reserved.

A Modern Vector Machine: Cray X1 (2002)



© 2007 Elsevier, Inc. All rights reserved.

Use of MSP:

1. An 8-lane SSP (SIMD)
2. 4 SSPs (MPMD or MIMD)
   - SPMD is a special case

GPU vs. Vector Processing

- Vector program
  - Scalar code + explicit vector operations
  - Efficient vector memory accessing instructions
  - Vector mask to handle control flow
- Vector processor complexity
  - Centralized multi-ported VRF
  - Precise interrupts
  - Low latency, high bandwidth memory system
- GPU program
  - Scalar code for each work item. Vectors (warps) are formed in hardware.
    - All code is vectorized, including scalar ones
  - Every memory access is treated as gather and scatter
    - Not efficient as LV, LVWS.
- GPU architecture complexity
  - Precise interrupts
  - Warp scheduler/thread block dispatcher: hardware version of strip-mining
  - Memory access unit due to gather and scatter, rediscovering the sequential pattern (coalescing)/stride pattern (prefetching)

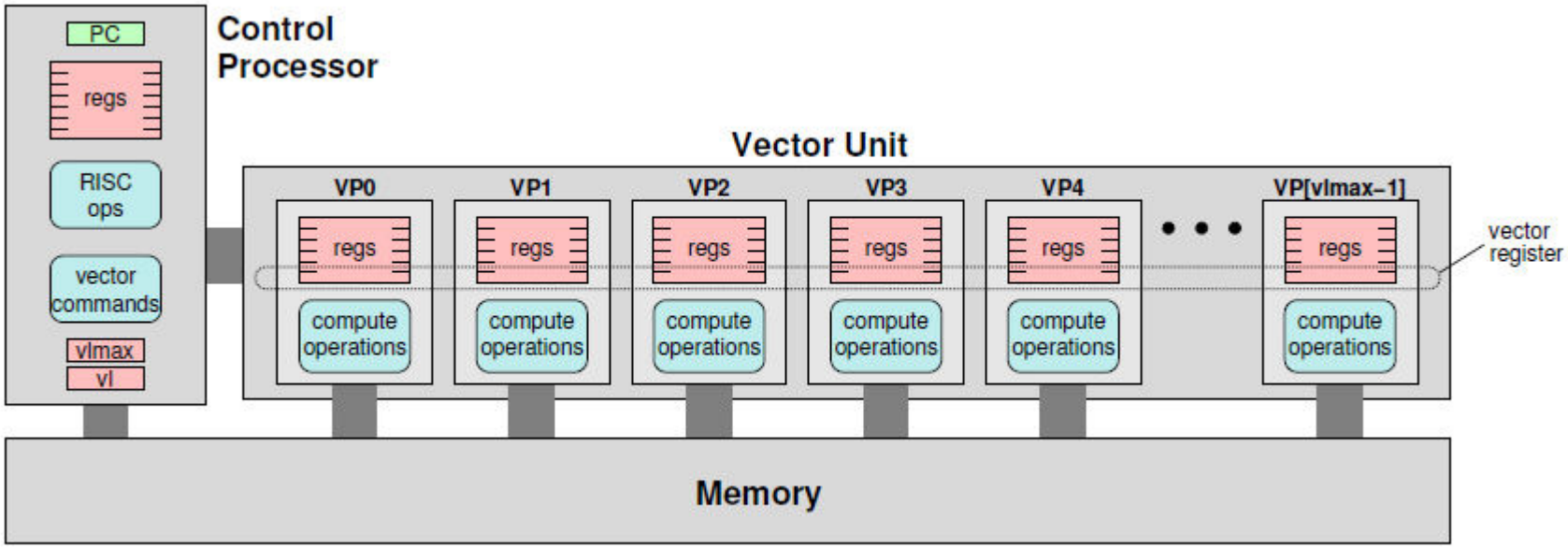
SIMD Instruction Set Extensions for Multimedia

- Compared to vector architecture
  - Fixed number of data operands (Number of data operands encoded into op code). No vector length register
  - No complex addressing modes, i.e., strided, gather-scatter
  - No mask register for conditional execution

SIMD example: DAXPY

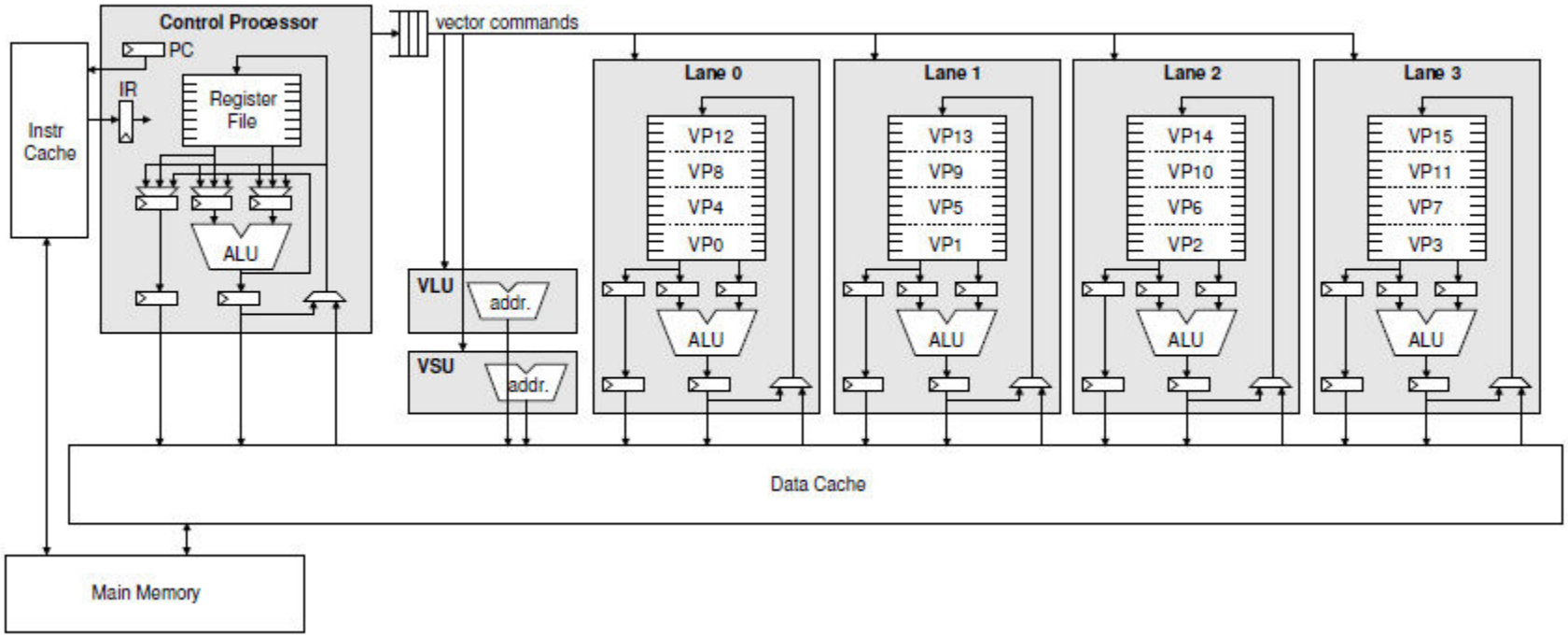
```
    L.D       F0,a          ;load scalar a
    MOV       F1, F0        ;copy a into F1 for SIMD MUL
    MOV       F2, F0        ;copy a into F2 for SIMD MUL
    MOV       F3, F0        ;copy a into F3 for SIMD MUL
    DADDIU    R4,Rx,#512    ;last address to load
Loop:L.4D     F4,0[Rx]      ;load X[i], X[i+1], X[i+2], X[i+3]
    MUL.4D    F4,F4,F0      ;a×X[i],a×X[i+1],a×X[i+2],a×X[i+3]
    L.4D      F8,0[Ry]              ;load Y[i], Y[i+1], Y[i+2], Y[i+3]
    ADD.4D    F8,F8,F4      ;a×X[i]+Y[i], ..., a×X[i+3]+Y[i+3]
    S.4D      0[Ry],F8              ;store into Y[i], Y[i+1], Y[i+2], Y[i+3]
    DADDIU    Rx,Rx,#32     ;increment index to X
    DADDIU    Ry,Ry,#32     ;increment index to Y
    DSUBU     R20,R4,Rx     ;compute bound
    BNEZ R20,Loop           ;check if done
```

Vector-Processor: Programmer's model



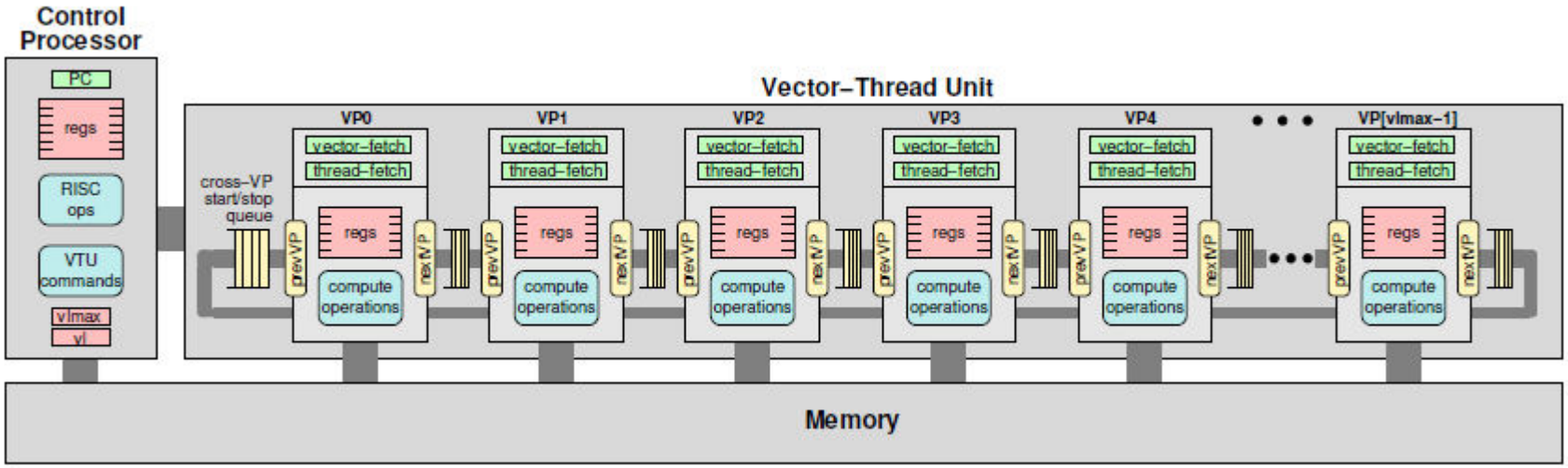
From R. Krashinsky's Ph.D. dissertation

Vector Processor: Architecture



From R. Krashinsky's Ph.D. dissertation

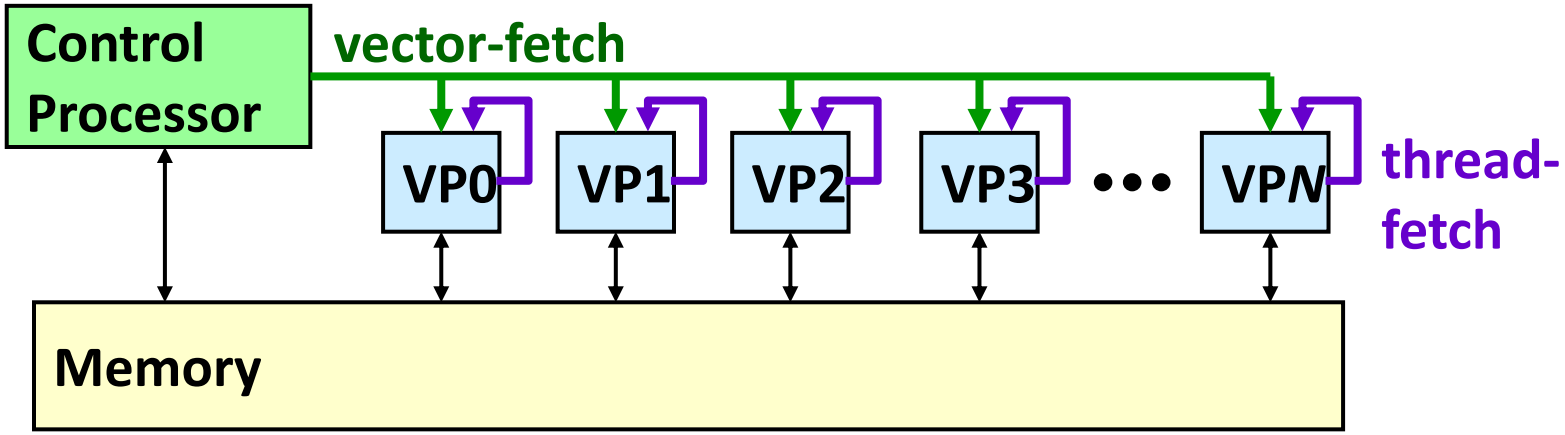
Vector-Thread Processors: Programmer's model



From R. Krashinsky's Ph.D. dissertation

Vector-Thread Architecture

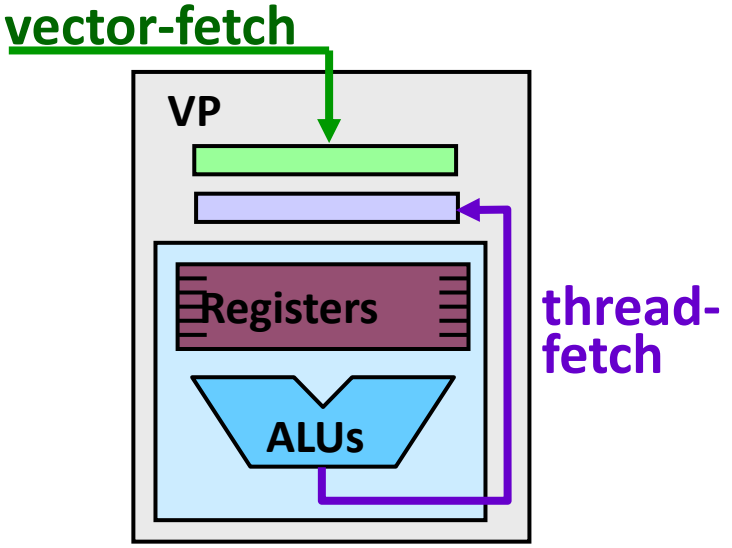
- VT unifies the vector and multithreaded compute models
- A control processor interacts with a vector of virtual processors (VPs)
- **Vector-fetch**: control processor fetches instructions for all VPs in parallel
- **Thread-fetch**: a VP fetches its own instructions
- VT allows a seamless intermixing of vector and thread control

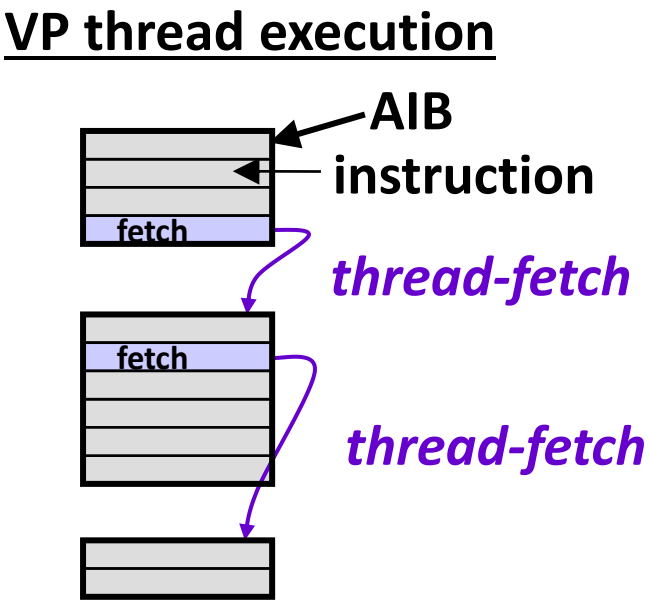


From VTA talk at ISCA'04

Virtual Processor Abstraction

- VPs contain a set of registers
- VPs execute RISC-like instructions grouped into atomic instruction blocks (AIBs)
- VPs have no automatic program counter, AIBs must be explicitly fetched
  - VPs contain pending vector-fetch and thread-fetch addresses
- A fetch instruction allows a VP to fetch its own AIB
  - May be predicated for conditional branch
- If an AIB does not execute a fetch, the VP thread *stops*

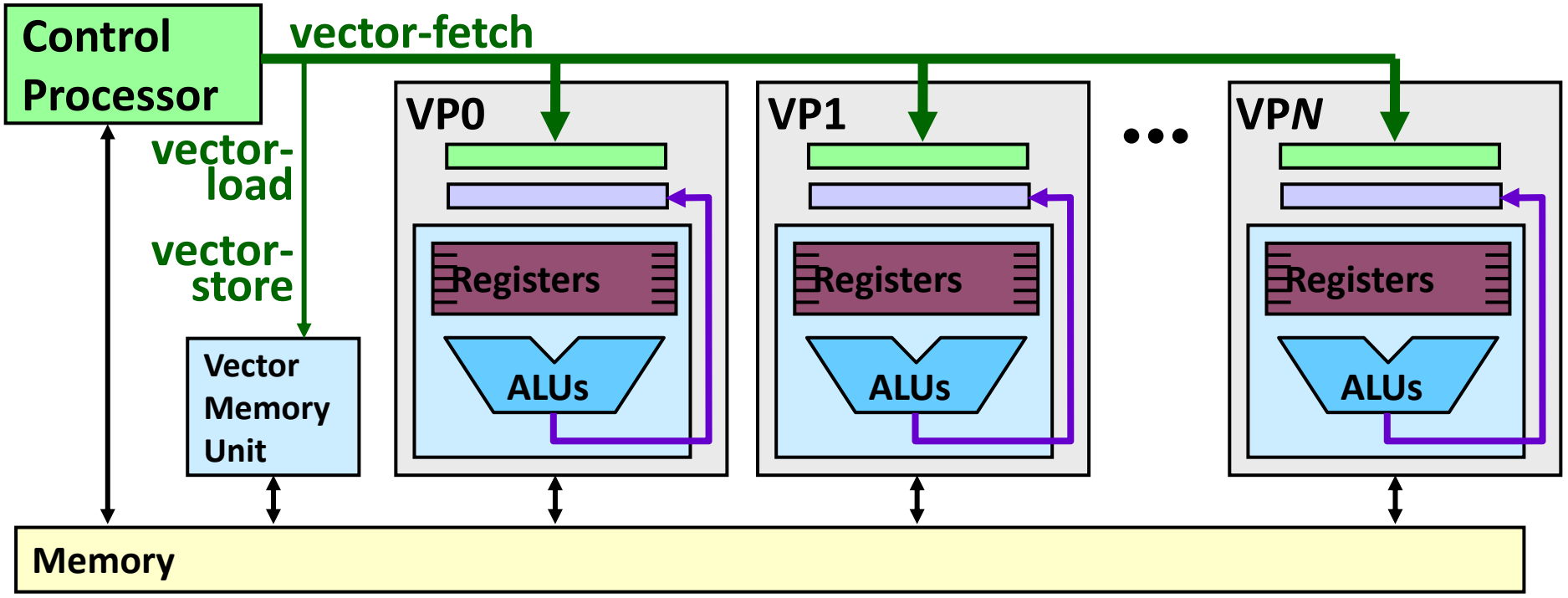




From VTA talk at ISCA'04

Virtual Processor Vector

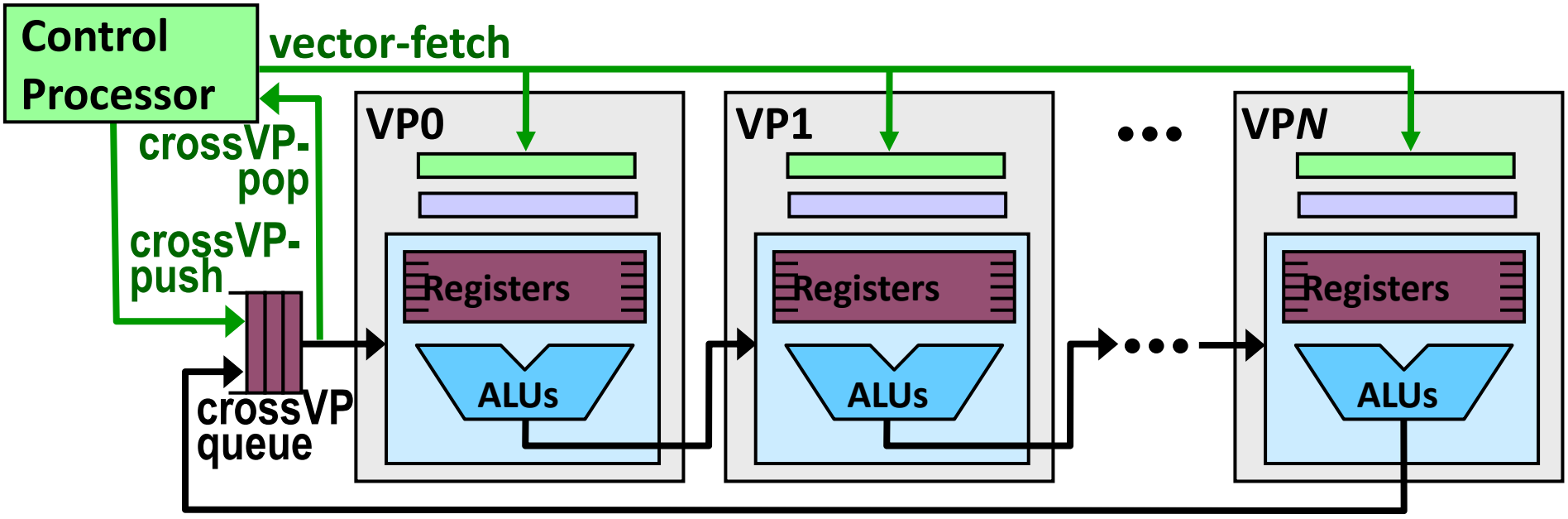
- A VT architecture includes a control processor and a virtual processor vector
  - Two interacting instruction sets
- A vector-fetch command allows the control processor to fetch an AIB for all the VPs in parallel
- Vector-load and vector-store commands transfer blocks of data between memory and the VP registers



From VTA talk at ISCA'04

Cross-VP Data Transfers

- Cross-VP connections provide fine-grain data operand communication and synchronization
  - VP instructions may target nextVP as destination or use prevVP as a source
  - CrossVP queue holds wrap-around data, control processor can push and pop
  - Restricted ring communication pattern is cheap to implement, scalable, and matches the software usage model for VPs



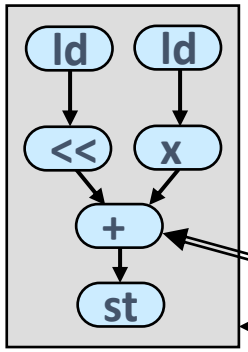
From VTA talk at ISCA'04

Mapping Loops to VT

- A broad class of loops map naturally to VT
  - Vectorizable loops
  - Loops with loop-carried dependencies
  - Loops with internal control flow
- Each VP executes one loop iteration
  - Control processor manages the execution
  - Stripmining enables implementation-dependent vector lengths
- Programmer or compiler only schedules one loop iteration on one VP
  - No cross-iteration scheduling

From VTA talk at ISCA'04

Vectorizable Loops



- Data-parallel loops with no internal control flow mapped using vector commands
  - predication for small conditionals

operation

loop iteration DAG

Control Processor

VP0 **VP1** **VP2** **VP3** ••• **VP*N***

vector-load ld ld ld ld ••• ld

vector-load ld ld ld ld ••• ld

vector-fetch

<< x + | << x + | << x + | << x + | ••• | << x +

vector-store st st st st ••• st

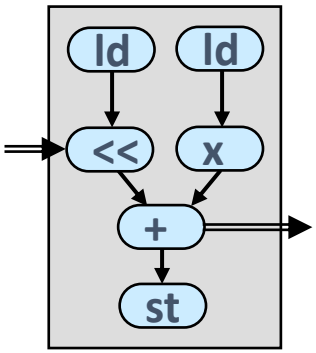
vector-load ld ld ld ld ••• ld

vector-load ld ld ld ld ••• ld

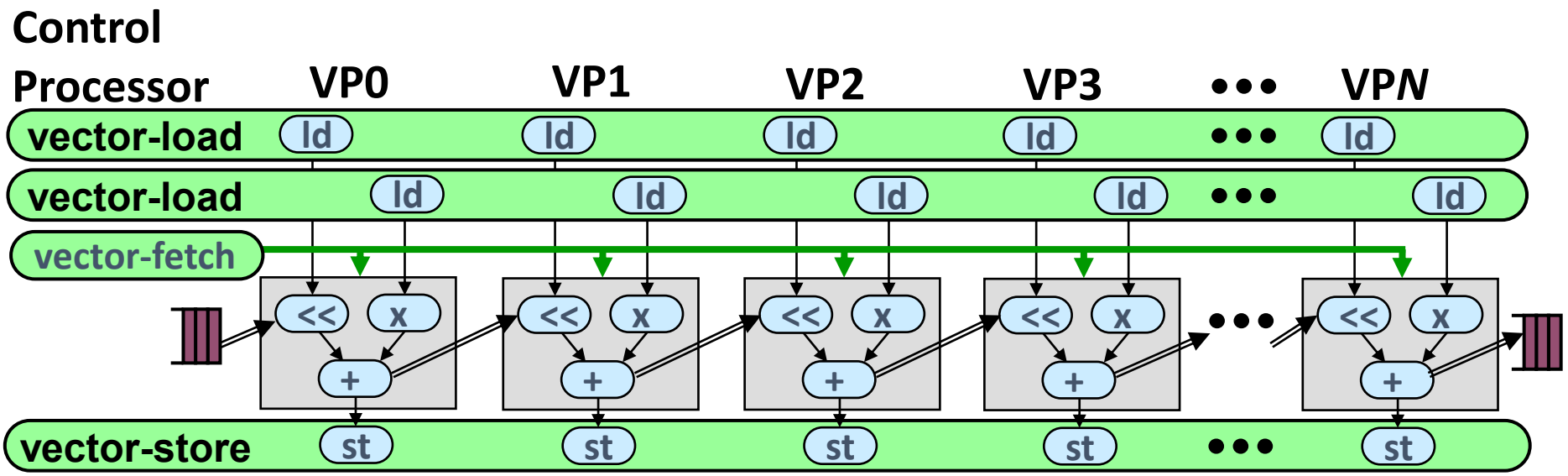
vector-fetch

From VTA talk at ISCA'04

Loop-Carried Dependencies

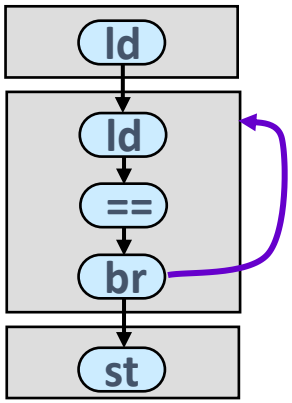


- Loops with cross-iteration dependencies mapped using vector commands with cross-VP data transfers
  - Vector-fetch introduces chain of prevVP receives and nextVP sends
  - Vector-memory commands with non-vectorizable compute

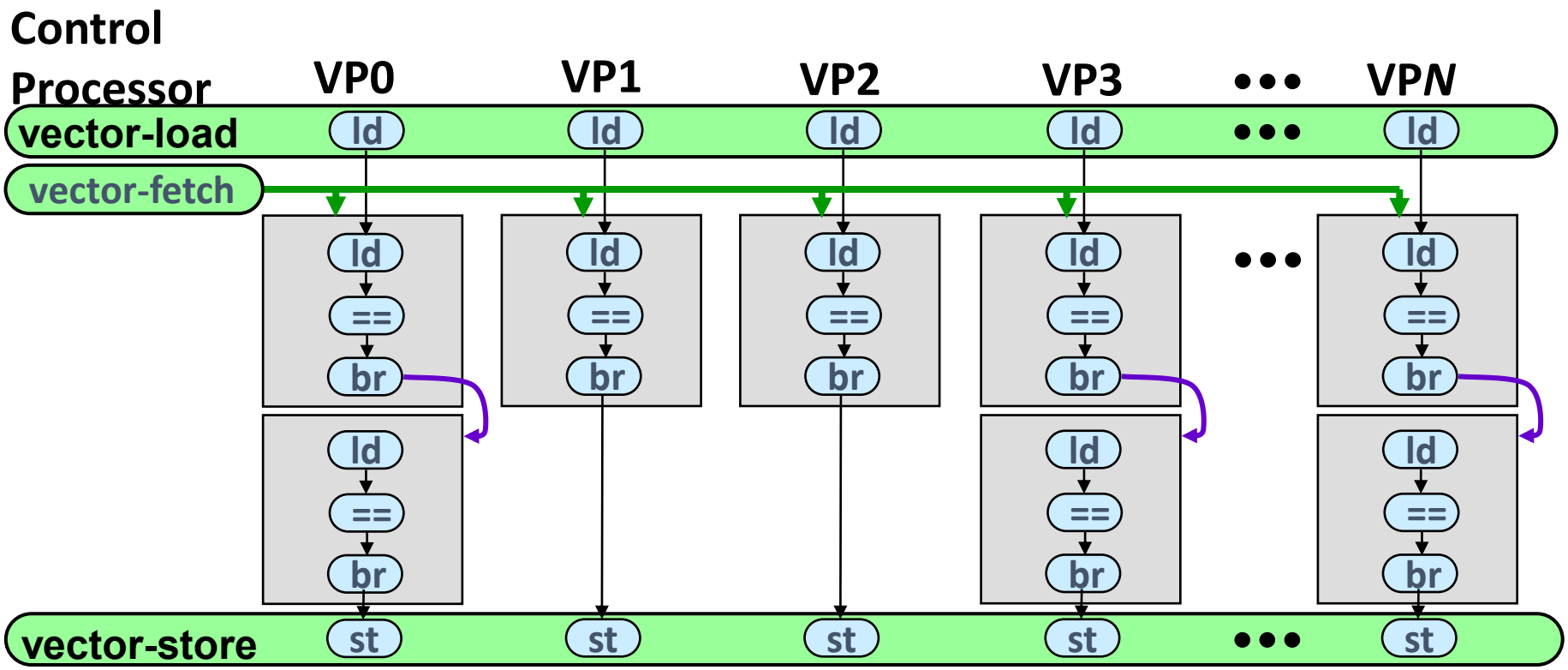


From VTA talk at ISCA'04

Loops with Internal Control Flow

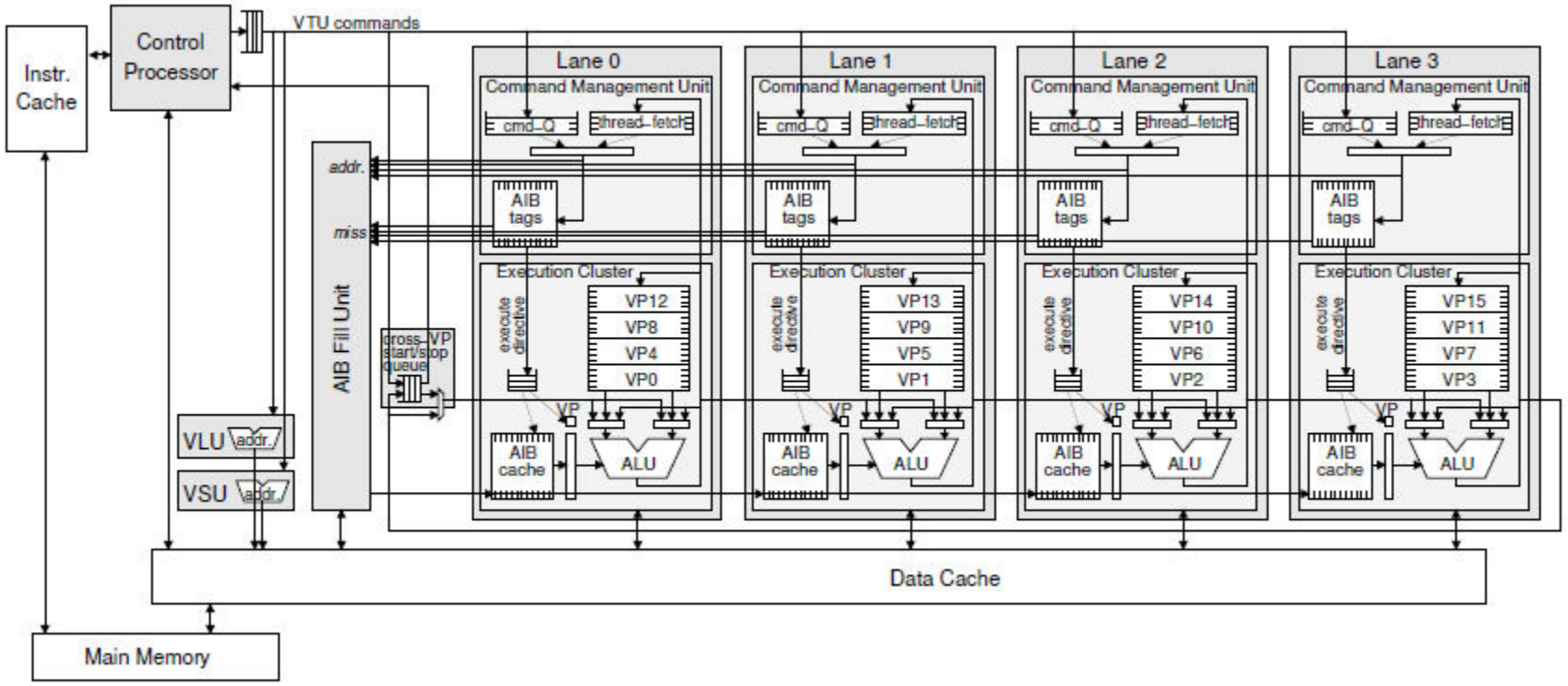


- Data-parallel loops with large conditionals or inner-loops mapped using thread-fetches
  - Vector-commands and thread-fetches freely intermixed
  - Once launched, the VP threads execute to completion before the next control processor command



From VTA talk at ISCA'04

Vector Thread Architecture



From R. Krashinsky's Ph.D. dissertation

VTA code example

- Linked list sea

```
void manySearch (int* in_ptr,
                 int* out_ptr,
                 Node* root,
                 int n) {
  for (int i=0; i<n; i++) {
    int searchv = *in_ptr++;
    Node* node = root;
    while(searchv != node->val)
      node = node->next;
    *out_ptr++ =  node->data;
  }
}
```

From R. Krashinsky's Ph.D. dissertation

VTA code example (cont.)

```
manySearch:
  vcfg  3, 3 # configure VPs: 3 pr, 3 sr
  sll   rstripstr, vlmax, 2
  la    r1, search_aib
  vwrsh sr0, r1     # sr0 = search_aib
  vwrsh sr1, root   # sr1 = root
stripmineloop:
  setvl vlen, n # (vl,vlen) = MIN(n,vlmax)
  vlw   pr1, rin     # vector-load searchv
  vf    init_aib     # setup node pointers
  vf    search_aib   # launch searches
  vf    output_aib   # get result data
  vsw   pr2, rout    # vector-store output
  add  rin,  rin,  rstripstr # incr rin
  add  rout, rout, rstripstr # incr rout
  sub  n, n, vlen            # decr n
  bnez n, stripmineloop      # stripmine
  jr   ra                    # func. ret.
```

```
init_aib:
 .aib begin
 copy pr0, sr1    # node = root
 .aib end

search_aib:
 .aib begin
 lw  sr2, 4(pr0) # load node->val
 seq p, sr2, pr1 # p := (searchv == node->val)
 (!p) lw pr0, 8(pr0)# if !p: node = node->next
 (!p) fetch sr0     # if !p: fetch search_aib
 .aib end

output_aib:
 .aib begin
 lw pr2, 0(pr0)  # load node->data
 .aib end
```

From R. Krashinsky's Ph.D. dissertation